Hafta 4 — Derinlemesine

> ## ⚠️ BU DOSYA SINAV KAPSAMI DIŞINDADIR
>
> Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, *"peki bu nasıl çalışıyor?"* diye soran öğrenci içindir. Sınav kapsamı `H04-ders-notu.md` dosyasıdır.
>
> Dönem projesinde işinize yarayacak birkaç şey de buradadır (bölüm 3, bölüm 6, bölüm 9).

---

1. Öncelik Sırası Nereden Geliyor?

Ders notu bölüm 3.2'de kaynak sırasını **Form → Rota → Sorgu** olarak verdik. Bu sıra sihirli değil; `MvcOptions.ValueProviderFactories` listesinin **sırasıdır**.

Çatı varsayılan olarak şu fabrikaları bu sırayla kaydeder:

#	Fabrika	Okuduğu yer
1	`FormValueProviderFactory`	`Request.Form`
2	`RouteValueProviderFactory`	`RouteData.Values`
3	`QueryStringValueProviderFactory`	`Request.Query`
4	`JQueryFormValueProviderFactory`	jQuery'nin ürettiği `a[0].b` biçimi

Model binding, parametre için değer ararken listeyi **baştan sona** gezer ve **ilk bulduğunu** alır. Sıra buradan çıkıyor.

İsterseniz değiştirebilirsiniz — ama çok iyi bir sebebiniz yoksa yapmayın:

```
builder.Services.Configure<MvcOptions>(o =>
{
    // query string'i en öne almak: artık ?deger= form gövdesini EZER
    var sorgu = o.ValueProviderFactories
        .First(f => f is QueryStringValueProviderFactory);
    o.ValueProviderFactories.Remove(sorgu);
    o.ValueProviderFactories.Insert(0, sorgu);
});
```

> ⚠️ Bunu yaparsanız projedeki **her** action'ın davranışı değişir ve hiçbir yerde yazmaz. Tek bir parametre için kaynağı zorlamak gerekiyorsa `[FromQuery]` kullanın.

`[FromQuery]` gibi attribute'ler bu listeyi **atlar**: parametreye tek bir `IValueProvider` bağlar.

---

2. `StringValues` ve `IFormCollection`

`IFormCollection["Ad"]` bir `string` değil, `StringValues` döndürür.

Sebebi: HTTP aynı adı **birden çok kez** taşıyabilir.

```
Etiket=roman&Etiket=klasik&Etiket=rusca
```

```
StringValues v = form["Etiket"];
v.Count;            // 3
v[0];               // "roman"
v.ToString();       // "roman,klasik,rusca"  ← virgülle birleştirir
(string?)v;         // aynı: birleştirilmiş metin
```

🦎 **Sessiz tuzak:** `form["Etiket"]` sonucunu doğrudan bir `string`e atarsanız üç değer **virgülle birleşmiş tek metne** dönüşür ve bunu fark etmeniz zor olur. Çoklu değer bekliyorsanız `StringValues` olarak tutun ya da model binding kullanıp `List<string>`'e bağlayın.

`StringValues` ayrıca **boş** olabilir: alan gönderilmemişse `v.Count == 0` ve `v.ToString()` boş metindir. "Gönderilmedi" ile "boş gönderildi" ayrımını `form.ContainsKey("Ad")` ile yaparsınız.

---

3. Seçici Binding: `[Bind]`, `[BindNever]`, `TryUpdateModelAsync`

Ders notu bölüm 4.4 ViewModel'i önerdi. Alternatifler de var; hepsinin bir bedeli var.

`[BindNever]` — özellik düzeyinde kapatmak

```
public class Kitap
{
    public string Baslik { get; set; } = "";

    [BindNever]                       // model binding bu alana ASLA dokunmaz
    public bool Onayli { get; set; }
}
```

Avantajı: kural **entity'nin yanında** durur; yeni bir action yazan kişi kuralı taşımayı unutamaz. **Dezavantajı:** aynı entity bir yönetim ekranında *gerçekten* bağlanmak istiyorsa orada da çalışmaz; ayrı bir yol gerekir.

`[Bind]` — izin listesi

```
[HttpPost]
public IActionResult Ekle([Bind("Baslik,Yazar,Fiyat")] Kitap kitap) { … }
```

Dezavantajı — ders notunda değinildi: entity'ye `Rol` alanı eklediğinizde bu listeyi güncellemeniz gerekmez (yeni alan bağlanmaz, iyi) **ama** `Yazar` alanının adını değiştirdiğinizde liste sessizce eskir ve o alan artık bağlanmaz. Kırılganlık burada: **yazım hatası derleme zamanında yakalanmaz.**

`TryUpdateModelAsync` — çalışma zamanında seçmek

```
var kitap = _depo.Bul(id)!;
await TryUpdateModelAsync(kitap, "", k => k.Baslik, k => k.Yazar, k => k.Fiyat);
```

İzin verilen alanlar **lambda ile**, yani **derleyicinin denetlediği** biçimde yazılır. Alan adı değişirse kod **derlenmez**. Güncelleme (edit) senaryolarında en sağlam yöntem budur.

> **Özet tercih sırası:** yeni kayıt için **ViewModel**; var olan kaydı güncellemek için `TryUpdateModelAsync`; `[Bind("…")]` metin listesinden kaçının.

---

4. Özel Model Bağlayıcı (custom model binder)

Bazen gelen metnin nesneye çevrilmesi standart kurallara sığmaz. Örnek: tek bir alanda `"978-0-14-044913-6"` gibi bir ISBN gelsin ve bunu doğrulanmış bir `Isbn` değer nesnesine çevirmek isteyelim.

```
public class IsbnBinder : IModelBinder
{
    public Task BindModelAsync(ModelBindingContext ctx)
    {
        var ham = ctx.ValueProvider.GetValue(ctx.ModelName).FirstValue;

        if (string.IsNullOrWhiteSpace(ham))
            return Task.CompletedTask;              // boş: dokunma, null kalsın

        if (!Isbn.TryParse(ham, out var isbn))
        {
            ctx.ModelState.TryAddModelError(ctx.ModelName, "Geçersiz ISBN.");
            return Task.CompletedTask;              // ⚠️ istisna FIRLATMA
        }

        ctx.Result = ModelBindingResult.Success(isbn);
        return Task.CompletedTask;
    }
}
```

Binding:

```
[ModelBinder(BinderType = typeof(IsbnBinder))]
public Isbn? Isbn { get; set; }
```

> **Değişmeyen kural:** binder **istisna fırlatmaz**. Geçersiz girdi bir uygulama durumudur; hatayı `ModelState`'e yazar ve kararı action'a bırakır. Ders notu bölüm 3.7'deki davranış tam olarak budur.

---

5. Kültüre Bağlı Çeviri: Ondalık Ayracı ve Tarih

Bu, Türkçe kullanan ekiplerin en sık çarptığı duvardır.

`decimal Fiyat` alanına kullanıcı `12,50` yazdı. Bağlanır mı?

Cevap: sunucunun kültürüne bağlı. Model binding, form ve sorgu değerlerini `CultureInfo.CurrentCulture` ile çevirir:

Sunucu kültürü	`12,50`	`12.50`
`tr-TR`	✅ 12,50	⚠️ **1250**
`en-US`	⚠️ **1250**	✅ 12.50
`InvariantCulture`	⚠️ **1250**	✅ 12.50

Tablo Hafta 6'da ölçüldü. Hiçbir hücrede "bağlanmadı" yok: yanlış ayraç **binlik ayracı** sayılıyor ve sayı hata vermeden büyüyor. Bu yüzden sorun fark edilmeden veritabanına kadar gidebilir.

Üstelik aynı uygulama **geliştirme makinenizde** çalışıp **sunucuda** çalışmayabilir.

İki yaklaşım:

1 — Kültürü açıkça sabitlemek (istenen davranış neyse onu seçin):

```
var kultur = new[] { new CultureInfo("tr-TR") };
app.UseRequestLocalization(new RequestLocalizationOptions
{
    DefaultRequestCulture = new RequestCulture("tr-TR"),
    SupportedCultures = kultur,
    SupportedUICultures = kultur
});
```

2 — Sayıyı HTML tarafında standartlaştırmak: `<input type="number">` kullanın. Tarayıcı değeri **noktalı** (invariant) biçimde gönderir; kullanıcı ekranda virgül görse bile.

> ⚠️ İkinci yol birinciyle **birlikte kullanılamaz**: sunucu `tr-TR`'ye sabitlenmişse tarayıcının gönderdiği `12.5` **125** olarak bağlanır. Hafta 6'da proje birinci yolu seçiyor; ölçümler ve gerekçe `H06-detay.md` bölüm 6'da.

> 🦎 **Rota ve query string için ayrı bir not:** ASP.NET Core rota değerlerini **invariant** kültürle çevirir, form değerlerini **CurrentCulture** ile. Yani aynı sayı `/Urun/Fiyat/12.50` yolunda çalışıp form gövdesinde çalışmayabilir. Tarih alanlarında bu fark baş ağrıtır; tarihleri `<input type="date">` ile alıp `yyyy-MM-dd` biçiminde göndermek en sağlamıdır.

6. `TempData` Sağlayıcısını Değiştirmek

Ders notu bölüm 1.2'de `TempData`'nın değeri bir **cookie**'ye yazdığını ve yönlendirmeden sağ çıktığını gördük. Burada bunun *nasıl* olduğuna bakıyoruz.

`TempData`'nın kendisi bir depo değildir; işi bir `ITempDataProvider`'a devreder. Varsayılan `CookieTempDataProvider`'dır — bu yüzden `AddSession` çağırmadan çalışır:

```
Set-Cookie: .AspNetCore.Mvc.CookieTempDataProvider=CfDJ8A4-mf96z...; path=/; samesite=lax; httponl
```

Değer **Data Protection** ile şifrelenir, ama yine de **her istekte** tarayıcıyla sunucu arasında gidip gelir. Pratik sonuçları:

Sonuç	Neden
Yalnızca serileştirilebilir basit tipler	Cookie bir metin taşır; `DefaultTempDataSerializer` kendi sınıfınızı çeviremez
Küçük tutulmalı	Cookie'ler için ~4 KB'lık tarayıcı sınırı vardır
Oturum gerekmez	Veri sunucuda değil, istemcide durur

Veriyi istemciye göndermek istemiyorsanız sağlayıcıyı oturum tabanlısıyla değiştirebilirsiniz:

```
builder.Services.AddControllersWithViews()
       .AddSessionStateTempDataProvider();   // sağlayıcıyı değiştiren satır
builder.Services.AddSession();
…
app.UseRouting();
app.UseAuthorization();
app.UseSession();                            // UseRouting'den SONRA
app.MapControllerRoute(…);
```

Bu durumda `TempData`'nın **API'si değişmez** — `Peek`, `Keep`, okununca silinme aynı kalır; yalnızca değerin durduğu yer değişir.

`Keep` ile `Peek` arasındaki gerçek fark

İkisi de değeri bir istek daha yaşatır, ama **zamanlaması** farklıdır:

	Ne yapar
`Peek("K")`	Okur ve **okundu işaretini hiç koymaz**
`Keep("K")`	Zaten okunmuş olanın **silinme işaretini kaldırır**

Yani `Keep`, değeri okuduktan *sonra* "yine de kalsın" demenin yoludur:

```
var mesaj = TempData["Mesaj"];   // okundu → silinecek
TempData.Keep("Mesaj");          // vazgeçtim, bir istek daha dursun
```

Nerede durmalı: `TempData` mı `Session` mı?

Taşıyıcı	Ömrü	Nerede durur	Ne zaman
`ViewData` / `ViewBag`	Tek istek	İstek nesnesi	Controller → view
`TempData`	Sonraki isteğe kadar	Cookie (varsayılan)	Yönlendirme sonrası mesaj
`Session`	Oturum boyunca	Sunucu (+ kimlik cookie'si)	Sepet, çok adımlı form

Sınır basittir: **tek bir yönlendirmeyi** aşması gereken veri `TempData`'ya, **birden çok sayfayı** aşması gereken veri `Session`'a gider.

---

7. CSRF — Neden Bu Hafta Yok?

Formunuzu başka bir sitedeki görünmez bir form **sizin adınıza** gönderebilir: kullanıcı sitenizde oturum açmışsa tarayıcı cookie'yi otomatik ekler ve sunucu isteği meşru sanır. Bu saldırının adı **CSRF (Cross-Site Request Forgery)**.

Savunma, forma sunucunun ürettiği gizli bir **anti-forgery token** koymak ve POST'ta doğrulamaktır:

```
<form method="post">
    @Html.AntiForgeryToken()
    …
</form>
```

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Ekle(KitapEkleVM girdi) { … }
```

Bu hafta konu edilmedi, çünkü **Tag Helper ile yazılan form bu token'ı kendiliğinden ekler** — yani asıl anlatım yeri Hafta 5'tir. Bu haftanın formlarını **elle** yazdığımız için token yok; ev görevinizde de beklenmiyor.

> 🧩 Yine de bu haftanın ilkesiyle aynı aileden: *istemciden gelen hiçbir veri güvenilir değildir* — isteğin **kaynağı** da dahil.

Kaynak: Prevent XSRF/CSRF attacks in ASP.NET Core

---

8. `[FromBody]` ve JSON — Hafta 12'nin Ön Tadı

Form gövdesi `ad=değer&…` biçimindeydi. Web API'lerde gövde genellikle **JSON**'dur ve onu okuyan attribute `[FromBody]`'dir.

```
[HttpPost]
public IActionResult Ekle([FromBody] KitapEkleVM girdi) { … }
```

```
POST /api/kitap
Content-Type: application/json

{ "baslik": "Suc ve Ceza", "fiyat": 120 }
```

Üç fark:

	Form binding	**`[FromBody]`**
Gövde biçimi	`x-www-form-urlencoded`	`application/json`
Okuyan	Value provider'lar	**Input formatter** (JSON çözücü)
Kaç kez	Kaynak sırası boyunca	Action'da **yalnızca bir** parametre
Ad eşleşmesi	Tam ad	Varsayılan olarak **büyük/küçük harf duyarsız**

> `[FromBody]`'nin tek kez kullanılabilmesinin sebebi teknik: gövde **ileriye doğru okunan bir akıştır**; ikinci parametre için baştan okunamaz. (Bu hafta `/Form` sayfasında gövdeyi iki kez okuyabildik çünkü `EnableBuffering()` çağırdık — yalnızca öğretim amaçlı bir istisna.)

---

9. Controller Dışından `HttpContext`: `IHttpContextAccessor`

Ders notu bölüm 2.6'da `HttpContext`'e controller içinden eriştik. Peki bir **servis sınıfından** aynı isteğe bakmak gerekirse?

Controller'daki `HttpContext` özelliği, çatının o istek için controller'a *verdiği* nesnedir. Kendi yazdığınız bir sınıfın böyle bir özelliği yoktur ve `HttpContext`'i **kurucuya parametre olarak almak da yanlıştır** — servis uygulama ömrü boyunca yaşayabilir, `HttpContext` ise tek bir istek kadar.

Doğru yol `IHttpContextAccessor`'dır:

```
builder.Services.AddHttpContextAccessor();   // Program.cs — kaydı şart
```

```
public class ZiyaretKaydi
{
    private readonly IHttpContextAccessor _erisim;

    public ZiyaretKaydi(IHttpContextAccessor erisim) => _erisim = erisim;

    public string? SuAnkiYol()
        => _erisim.HttpContext?.Request.Path.Value;   // ? zorunlu
}
```

Dikkat edilecek üç nokta:

Nokta	Neden
`HttpContext` **nullable**	Arka plan işi gibi istek dışı bağlamlarda `null`'dır
`AddHttpContextAccessor()` unutulmaz	Kaydedilmezse enjeksiyon hatası alırsınız
Ucuz değildir	`AsyncLocal` üzerinden çalışır; sıcak yollarda gereksiz kullanmayın

Daha temiz alternatif: servise `HttpContext`'i değil, ondan **çıkardığınız değeri** geçmek. Controller zaten isteği görüyor:

```
// controller
_kayit.Yaz(Request.Path, User.Identity?.Name);
```

Böylece servis test edilebilir kalır: `HttpContext` kurmadan çağırabilirsiniz. `IHttpContextAccessor`, bu geçiş mümkün olmadığında başvurulacak araçtır.

---

10. Daha İleri Gitmek İsteyenlere

Konu	Nerede
Model binding — tam referans	https://learn.microsoft.com/aspnet/core/mvc/models/model-binding?view=aspnetcore-10.0
Custom model binder	https://learn.microsoft.com/aspnet/core/mvc/advanced/custom-model-binding?view=aspnetcore-10.0
CSRF koruması	https://learn.microsoft.com/aspnet/core/security/anti-request-forgery?view=aspnetcore-10.0
İstek yerelleştirme (kültür)	https://learn.microsoft.com/aspnet/core/fundamentals/localization?view=aspnetcore-10.0
Dosya yükleme	https://learn.microsoft.com/aspnet/core/mvc/models/file-uploads?view=aspnetcore-10.0
Form verisi gönderme (MDN)	https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Sending_and_retrieving_form_data

> ⚠️ Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır.** Sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür ve `Startup.cs` 'li örneklerle karşılaşırsınız.